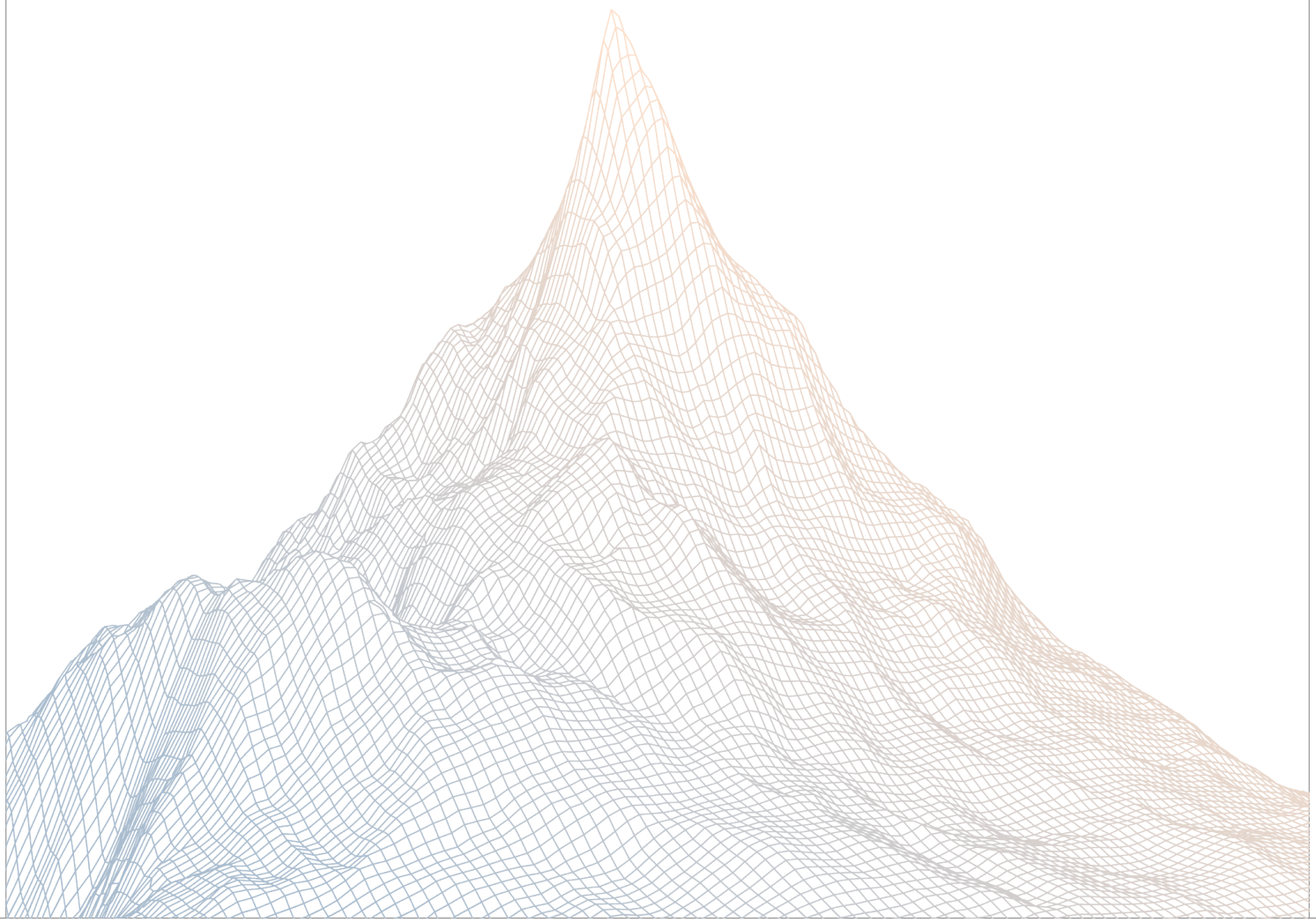
Zenith

StarkWare

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: May 5th to May 15th, 2025

AUDITED BY: EV_om
Mario Poneder

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About StarkWare 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 7

4.1 Low Risk 8

4.2 Informational 19

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About StarkWare

StarkWare develops STARK-based solutions for the blockchain industry. Our products facilitate secure, trustless, and scalable blockchain applications.

2.2 Scope

The engagement involved a review of the following targets:

Target	sn-lbtc
Repository	https://github.com/starkware-libs/sn-lbtc
Commit Hash	`9630630a11d055e621d5b18e7351b034311fc518`
Files	`Consortium Bascule and LBTC contracts.`

2.3 Audit Timeline

May 5th, 2025	Audit start
May 15th, 2025	Audit end
June 5th, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	9
Informational	8
Total Issues	**17**

3

Findings Summary

ID	Description	Status
L-1	The threshold_weight can be set to an arbitrarily low non-zero value undermining security	Acknowledged
L-2	Deposits can be prevented from being correctly reported by a malicious validation consumer	Acknowledged
L-3	The withdrawal_amount can be under- or overreported by a malicious validation consumer	Acknowledged
L-4	Emitted DepositsReported includes skipped deposits	Acknowledged
L-5	The set_next_validator_set transaction could be replayed across networks	Acknowledged
L-6	Missing functionality to revoke and add permitted minters after initialization	Resolved
L-7	Admin functions unnecessarily restricted by paused state	Acknowledged
L-8	No mechanism to revert incorrectly reported deposits	Resolved
L-9	Missing ERC20Permit implementation breaks feature parity with Solidity version	Resolved
I-1	Validator weights are not checked to be in reasonable proportion allowing centralization	Acknowledged
I-2	A MIN_VALIDATOR_SET_SIZE of 1 bears centralization risks	Acknowledged
I-3	Missing validation for report_id event key	Acknowledged
I-4	Pausing the lbtc contract also pauses permissioned mints and burns	Acknowledged
I-5	The burn commission can be set arbitrarily high by the governor	Resolved

ID	Description	Status
I-6	External integrations will fail due to missing SRC5 interface	Acknowledged
I-7	The MAX_VALIDATOR_SET_SIZE constant in the Cairo contract deviates from the Solidity version	Acknowledged
I-8	The check_proof function skips invalid signatures and allows duplicates on early return	Acknowledged

4 Findings

4.1 Low Risk

A total of 9 low risk findings were identified.

[L-1] The `threshold_weight` can be set to an arbitrarily low non-zero value undermining security

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- packages/consortium/src/consortium.cairo#L230-L232

Description:

The `threshold_weight` parameter in the `set_next_validator_set` function can be set to an arbitrarily low non-zero value by the current validators. This allows proof verification in the next epoch to succeed with as little as one signature, significantly reducing the security and decentralization of the protocol. A malicious or colluding validator set could exploit this to gain control over the system with minimal participation, undermining the integrity of the consensus mechanism.

Recommendations:

It is recommended to enforce a minimum `threshold_weight` requirement of at least 50% of the validator set's `total_weight`.

StarkWare: We are aware of this possibility, but do not consider it as a serious issue, this behavior is the same as implemented by EVM smart-contracts. We prefer to keep it this way.

Zenith: Acknowledged.

[L-2] Deposits can be prevented from being correctly reported by a malicious validation consumer

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- packages/bascule/src/bascule.cairo#L191-L213
- packages/bascule/src/bascule.cairo#L114-L163

Description:

A malicious `validation_consumer` can exploit the `validate_withdrawal` function to front-run the `report_deposits` function. By calling `validate_withdrawal` with withdrawal amounts below the validation threshold, the deposit's state is directly set to `DepositState::Withdrawn` instead of `DepositState::Reported`. This prevents the deposit from being properly recorded in the `report_deposits` process, leading to incorrect accounting and potential loss of funds. Additionally, there are no restrictions on which deposit IDs can be validated, allowing non-existent or to-be-reported deposits being set to `DepositState::Withdrawn`.

Recommendations:

It is recommended to add validation to prevent non-existent or to-be-reported deposits from being preliminarily set to `DepositState::Withdrawn`.

StarkWare: We are aware about this possibility, but do not consider it a serious issue. This is intentional because we want to have information about all withdrawals to be present in smart-contract. This behavior is the same as implemented by EVM smart-contracts. We prefer to keep it this way.

Zenith: Acknowledged.

[L-3] The `withdrawal_amount` can be under- or overreported by a malicious validation consumer

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- packages/bascule/src/bascule.cairo#L114-L163

Description:

The `validate_withdrawal` function in the `bascule` contract allows the `withdrawal_amount` parameter to be provided by the `validation_consumer` without any on-chain validation. A malicious `validation_consumer` could exploit this by underreporting or overreporting the `withdrawal_amount`, leading to incorrect accounting of withdrawals. This could result in unauthorized withdrawals or discrepancies in the contract's state.

Recommendations:

It is recommended to implement the following mitigation measures:

1. Add on-chain validation to ensure the `withdrawal_amount` matches the actual deposit amount associated with the `deposit_id`.
2. Consider introducing cryptographic proofs to validate the `withdrawal_amount` provided by the `validation_consumer`.

StarkWare: We are aware of this possibility, but do not consider it a serious issue, this behavior is the same as implemented by EVM smart-contracts. We prefer to keep it this way.

Zenith: Acknowledged.

[L-4] Emitted `DepositsReported` includes skipped deposits

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- bascule.cairo

Description:

The `report_deposits()` function emits a `DepositsReported` event with an inaccurate count of successfully processed deposits:

```
self.emit(events::DepositsReported { report_id, num_deposits:
    deposit_ids_len })
```

The function reports all deposits that were attempted (`deposit_ids_len`), rather than only those successfully recorded. When deposits are already reported, they emit a `DepositAlreadyReported` event and are skipped:

```
if deposit_id_entry.read() == DepositState::Unreported {
    deposit_id_entry.write(DepositState::Reported);
} else {
    // Only warn instead of reverting.
    self.emit(events::DepositAlreadyReported { deposit_id: *deposit_id })
}
```

This discrepancy could mislead off-chain systems that rely on these events for monitoring, as they might incorrectly assume all reported deposits were newly registered.

Recommendations:

Track and report only successfully processed deposits.

Lombard: We are aware about this possibility, but do not consider it as a serious issue, this behavior is the same as implemented by EVM smart-contracts. We prefer to keep it this way.

Zenith: Acknowledged.

[L-5] The `set_next_validator_set` transaction could be replayed across networks

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- packages/consortium/src/consortium.cairo#L107-L125

Description:

A `set_next_validator_set` transaction from the testnet could potentially be replayed on the mainnet (if the next epoch matches) due to the lack of chain ID inclusion in the `msg_hash`. While the likelihood of this happening is extremely low due to the need for sufficient overlap in validator weights to pass `check_proof`, it remains a theoretical risk. This could lead to unintended validator set updates on the mainnet.

Recommendations:

It is recommended to include the chain ID in the `msg_hash` and proof generation to ensure that `set_next_validator_set` transactions are network-specific and cannot be replayed across networks.

StarkWare: This is theoretically feasible if both the following conditions hold: validators use the same keys on mainnet and testnet, and the different validators set have a common subset in the top-most epoch of one of the two which would overcome the weight threshold.

Mitigating this unlikely issue would mean to sign different validators set update payload for each chain, which involves additional effort on the notary and backend side which is overkilling for the low risk of the issue at this moment. Also, validators use different notary keys between testnet and mainnet as can be verified on the respective explorers mintscan - notary and mintscan - testnet notary, which prevents any replay risk.

Zenith: Acknowledged.

[L-6] Missing functionality to revoke and add permitted minters after initialization

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/token/src/lbtc/token.cairo#L156-L168

Description:

The `lbtc` contract lacks functionality to revoke permitted minters. This is critical in cases where the trust assumptions about a minter are no longer valid. Additionally, the contract does not allow adding new permitted minters after initialization, which limits flexibility and adaptability. In contrast, the Solidity version of the `LBTC` contract provides mechanisms to manage minters dynamically.

Recommendations:

It is recommended to implement a function to revoke permitted minters to ensure that trust assumptions can be updated as needed. Furthermore, it is beneficial to add functionality to allow the addition of new permitted minters after initialization, similar to the Solidity version of the `LBTC` contract.

StarkWare: Resolved with @0362b6289... by removing the permitted minters altogether.

Zenith: Verified.

[L-7] Admin functions unnecessarily restricted by paused state

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- bascule.cairo

Description:

In the `bascule.cairo` contract, the admin functions `update_validation_threshold()` and `set_max_deposits()` both call `self.pausable.assert_not_paused()`. This unnecessarily restricts these functions from being used during a paused state.

Since these functions can only be called by the app governor, the pause check may be redundant and prevents administrators from adjusting critical parameters during emergency situations.

Recommendations:

Consider removing the pause check from both functions.

Lombard: We prefer to pause as many functions as possible. So such behavior is intentional and is the same as implemented by EVM smart-contracts.

Zenith: Acknowledged.

[L-8] No mechanism to revert incorrectly reported deposits

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- bascule.cairo

Description:

The `bascule.cairo` contract lacks functionality to unreport or block deposits after they've been reported. Once the `report_deposits()` function changes a deposit's state to `Reported`, this action cannot be reversed, even by an admin.

This creates a security risk if an operator mistakenly reports incorrect deposits or is compromised.

While the contract can be paused to prevent new reports, existing incorrectly reported deposits remain valid for withdrawal with no way to invalidate them.

Recommendations:

Implement an admin function to manually set specific deposits to Unreported or Withdrawn.

Lombard: Fixed in https://github.com/starkware-libs/sn-lbtc/pull/179. In our impl - the "withdrawal validator" is the `operator` and the function report_deposits checks that `only_operator()` is the caller of the function. The role can have multiple accounts and add/remove.

Zenith: Confirmed. Noting that the `TOKEN_ADMIN` is now the role allowed to call `validate_withdrawal()`, which is different from the `OPERATOR` role calling `report_deposits()`. Both can be managed via `RolesComponent`.

[L-9] Missing ERC20Permit implementation breaks feature parity with Solidity version

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- token.cairo

Description:

The `token.cairo` contract does not implement the ERC20Permit functionality that is present in the Solidity version of the contract, breaking feature parity between implementations.

The `ERC20PermitImpl` is declared in the same OpenZeppelin file as `ERC20Impl`, but needs to be integrated explicitly.

Recommendations:

Integrate ERC20Permit as shown below:

```
-- a/packages/token/src/lbtc/token.cairo
++ b/packages/token/src/lbtc/token.cairo
@@ -6,6 +6,8 @@ mod lbtc {
     use openzeppelin::introspection::src5::SRC5Component;
     use openzeppelin::token::erc20::ERC20Component;
     use openzeppelin::token::erc20::interface::IERC20Metadata;
    use openzeppelin::utils::cryptography::nonces::NoncesComponent;
    use openzeppelin::utils::snip12::SNIP12Metadata;
     use starknet::storage::{Map, StorageMapReadAccess,
    StorageMapWriteAccess};
     use starknet::{ContractAddress, get_caller_address};
     use starkware_utils::components::pausable::PausableComponent;
@@ -17,6 +19,7 @@ mod lbtc {
     use starkware_utils::interfaces::mintable_token::IMintableToken;

     component!(path: ERC20Component, storage: erc20, event: ERC20Event);
```

```
    component!(path: NoncesComponent, storage: nonces, event: NoncesEvent);
     component!(path: AccessControlComponent, storage: accesscontrol,
    event: AccessControlEvent);
     component!(path: SRC5Component, storage: src5, event: SRC5Event);
     component!(path: RolesComponent, storage: roles, event: RolesEvent);
@@ -28,6 +31,9 @@ mod lbtc {
     #[abi(embed_v0)]
     impl ERC20Impl = ERC20Component::ERC20Impl<ContractState>;

    #[abi(embed_v0)]
    impl ERC20PermitImpl = ERC20Component::ERC20PermitImpl<ContractState>;

     #[abi(embed_v0)]
     impl ReplaceabilityImpl =
         ReplaceabilityComponent::ReplaceabilityImpl<ContractState>;
@@ -40,6 +46,7 @@ mod lbtc {

     // Internal
     impl ERC20InternalImpl = ERC20Component::InternalImpl<ContractState>;
    impl NoncesImpl = NoncesComponent::NoncesImpl<ContractState>;

     #[storage]
     struct Storage {
@@ -47,6 +54,8 @@ mod lbtc {
         #[substorage(v0)]
         erc20: ERC20Component::Storage,
         #[substorage(v0)]
        nonces: NoncesComponent::Storage,
        #[substorage(v0)]
         accesscontrol: AccessControlComponent::Storage,
         #[substorage(v0)]
         src5: SRC5Component::Storage,
@@ -66,6 +75,8 @@ mod lbtc {
         #[flat]
         ERC20Event: ERC20Component::Event,
         #[flat]
        NoncesEvent: NoncesComponent::Event,
        #[flat]
         AccessControlEvent: AccessControlComponent::Event,
         #[flat]
         SRC5Event: SRC5Component::Event,

@@ -116,6 +127,17 @@ mod lbtc {
```

```
        }
    }

impl SNIP12MetadataImpl of SNIP12Metadata {
    fn name() -> felt252 {
        let state = unsafe_new_contract_state();

        // Some logic to get the name from storage
        state.erc20.name().at(0).unwrap().into()
    }

    fn version() -> felt252 { 'v1' }
}

impl HooksImpl<> of ERC20Component::ERC20HooksTrait<ContractState> {
    fn before_update(
        ref self: ERC20Component::ComponentState<ContractState>,
```

Lombard: Fixed in PR-208

Zenith: Confirmed

4.2 Informational

A total of 8 informational findings were identified.

[I-1] Validator weights are not checked to be in reasonable proportion allowing centralization

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- packages/consortium/src/consortium.cairo#L107-L125

Description:

The current implementation does not enforce checks to ensure that the weights assigned to validators are in reasonable proportion. This allows the consortium to circumvent decentralization by assigning the majority of the weight to a single validator. Specifically, the case where the weight of a single validator exceeds the `threshold_weight` should be prevented, except in the scenario where there is only one validator in the set.

Recommendations:

It is recommended to add a validation step to ensure that no single validator's weight exceeds the `threshold_weight`, except when there is only one validator in the set.

StarkWare: We are aware of this possibility, but do not consider it a serious issue, this behavior is the same as implemented by EVM smart-contracts. We prefer to keep it this way.

Zenith: Acknowledged.

[I-2] A `MIN_VALIDATOR_SET_SIZE` of 1 bears centralization risks

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- packages/consortium/src/consortium.cairo#L30

Description:

The `MIN_VALIDATOR_SET_SIZE` constant is currently set to 1, which allows a validator set to consist of a single validator. This creates a centralization risk, as a single validator would have complete control over the consortium. Such a configuration undermines the security and decentralization principles of the protocol.

Recommendations:

It is recommended to increase the `MIN_VALIDATOR_SET_SIZE` to 4 to ensure a more decentralized and secure validator set.

StarkWare: We are aware of this possibility, but do not consider it a serious issue, this behavior is the same as implemented by EVM smart-contracts. We prefer to keep it this way.

Zenith: Acknowledged.

[I-3] Missing validation for `report_id` event key

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- bascule.cairo

Description:

The `report_deposits()` function emits a `DepositsReported` event with `report_id` as a key field without validating its uniqueness:

```
self.emit(events::DepositsReported { report_id, num_deposits:
    deposit_ids_len })
```

In `events.cairo`, the `report_id` is defined as a key:

```
pub struct DepositsReported {
    #[key]
    pub report_id: u256,
    pub num_deposits: u32,
}
```

According to the Cairo documentation:

> Each event data field can be annotated with the attribute `#[key]`. Key fields are then stored separately than data fields to be used by external tools to easily filter events on these keys.

Without validating uniqueness, duplicate `report_id` values could be used in different deposit reports, making it difficult for off-chain systems to correctly filter and process events.

Recommendations:

Use an auto-incrementing counter instead of accepting `report_id` as a parameter, or validate that the provided `report_id` hasn't been used before.

Lombard: We are aware about this possibility, but do not consider it as a serious issue, this behavior is the same as implemented by EVM smart-contracts. We prefer to keep it this way.

Zenith: Acknowledged.

[I-4] Pausing the `lbtc` contract also pauses permissioned mints and burns

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- packages/token/src/lbtc/token.cairo#L120-L127

Description:

The `before_update` hook in the `lbtc` contract calls `pausable.assert_not_paused()`, which pauses all token-related operations, including permissioned mints and burns. This behavior may not align with the intended functionality if the goal is to only pause token transfers. Pausing permissioned mints and burns could unnecessarily restrict administrative or governance operations.

Recommendations:

If the intention is to only pause token transfers, it is recommended to update the `before_update` hook to exempt permissioned mints and burns from the paused check.

StarkWare: We prefer to pause as many functions as possible. So such behavior is intentional and is the same as implemented by EVM smart-contracts.

Zenith: Acknowledged.

[I-5] The burn commission can be set arbitrarily high by the governor

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/bridge/src/bridge.cairo#L191-L194

Description:

The `change_burn_commission` function allows the governor to set the burn commission to an arbitrarily high value without any upper limit. This makes it impossible for users to estimate the worst-case scenario for fees, potentially undermining trust in the protocol and discouraging participation.

Recommendations:

It is recommended to introduce an upper limit for the burn commission to ensure it remains within a reasonable range.

StarkWare: Fixed in @166787ade...

Zenith: Verified.

[I-6] External integrations will fail due to missing SRC5 interface

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- token.cairo
- consortium.cairo
- bascule.cairo

Description:

The SRC-5 standard provides a mechanism for contracts to advertise which interfaces they support, allowing other contracts to adapt their behavior accordingly.

All three main contracts in the codebase (`bascule.cairo`, `token.cairo`, and `consortium.cairo`) declare the SRC5 component, storage, and events:

```
component!(path: SRC5Component, storage: src5, event: SRC5Event);
```

However, none of these contracts actually expose the SRC5 interface to the public by embedding the implementation:

```
#[abi(embed_v0)]
impl SRC5Impl = SRC5Component::SRC5Impl<ContractState>;
```

This can be verified with a quick test:

```
#[test]
fn test_lbtc_snip5_support() {
    // This test verifies SNIP-5 support
    // SNIP-5:
    https://github.com/starknet-io/SNIPs/blob/main/SNIPS/snip-5.md

    let lbtc_address = init_token_with_roles();

    // Check if the contract supports SRC5 interface
    let src5 = ISRC5Dispatcher { contract_address: lbtc_address };
```

```
    let supports_src5 = src5.supports_interface(ISRC5_ID);
    assert!(supports_src5, "Should support SRC5");
}
```

Without proper SRC-5 implementation, other contracts cannot reliably determine whether these contracts support specific interfaces.

Recommendations:

Add the missing declaration to each contract.

Additionally, during initialization, each contract should register the interfaces it implements. For example, in the `token.cairo` contract's constructor:

```
self.src5.register_interface(IERC20_ID);
```

Note that some contracts already register their interfaces on initialization, e.g. `AccessControlComponent::initializer()`, called from `RolesComponent.initialize()`.

Lombard: Won't fix.

Zenith: Acknowledged.

[I-7] The `MAX_VALIDATOR_SET_SIZE` constant in the Cairo contract deviates from the Solidity version

`SEVERITY:` Informational	`IMPACT:` Informational
`STATUS:` Acknowledged	`LIKELIHOOD:` Low

Target

- packages/consortium/src/consortium.cairo#L31

Description:

The `MAX_VALIDATOR_SET_SIZE` constant in the Cairo contract is set to 25, whereas in the Solidity version it is set to 102.

Recommendations:

It is recommended to document the rationale for the chosen `MAX_VALIDATOR_SET_SIZE` in the Cairo contract.

StarkWare: The difference in `MAX_VALIDATOR_SET_SIZE` value between EVM and Starknet versions is due to the blockchain limits and gas consumption. We are aware about it and prefer to keep it the way it is now.

Zenith: Acknowledged.

[I-8] The `check_proof` function skips invalid signatures and allows duplicates on early return

`SEVERITY:` Informational	`IMPACT:` Informational
`STATUS:` Acknowledged	`LIKELIHOOD:` Low

Target

- packages/consortium/src/consortium.cairo#L127-L158

Description:

The `check_proof` function skips invalid signatures and allows duplicates after the `total_weight ⩾ threshold_weight` condition is met, causing an early return. While this behavior is acceptable in the current context, it may lead to issues in external integrations where stricter validation is required. Specifically, invalid signatures are ignored, and duplicate signatures may go undetected if the threshold is reached early.

Recommendations:

It is recommended to implement a strict version of `check_proof` for external integrations that:

- Validates all signatures, ensuring no invalid signatures are present.
    - Iterates over all signatures to detect duplicates, even if the `threshold_weight` is reached early.

Furthermore, it is advisable to update the documentation to clarify the behavior of the current `check_proof` function and the purpose of the stricter version.

StarkWare: Currently, both EVM smart-contract and Starknet contract skip invalid (and duplicating) signatures. We prefer the logic to be this way.

Zenith: Acknowledged.